

ECE 533 Final Exam 2026

Assigned: Saturday Morning, May 9th, 7 am, 2026.

Due: Sunday, May 10th, Midnight, 2026.

You must submit your final exam online.

No collaboration is allowed.

You must direct all questions to the instructor ONLY.

The contribution of the final exam to your final grade will be calculated based on the syllabus.

1 Grading

You will receive zero credit if your answers appear to be AI-generated. During grading, I will try to assess your understanding of the material as opposed to text that appears to be AI-generated. The credit is **not** distributed equally among problems.

We will meet at our usual time at 9 am for review. The Zoom link for the review is <https://unm.zoom.us/j/922>

After the review, I will upload the Zoom recording to the assignment link. In other words, in order to access the recording, you will need to login in the same place where you downloaded the final exam.

After Monday, I will send everyone information about the final project.

Problem #1. The cross-entropy loss function (10 points).

The expression for the cross-entropy function is given by:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(\hat{p}_k^{(i)}(\Theta) \right)$$

where Θ denotes the parameters learned by the network. Here, k is an index for each of the K possible classes, i refers to the i -th sample in a batch of m samples, $y_k^{(i)}$ is the ground truth for the k -th class for the i -th sample. Based on the learned parameters, $\hat{p}_k^{(i)}$ denotes the estimated probability estimated by the network for the i -th sample belonging to the k -th class. Note that $0 \log(0) = 0$ as it can be shown by L'Hopital's rule.

For each case below, you need to provide:

1. ground truth vectors.
2. probability values (not necessarily ideal) to be learned by the network.
3. cross-entropy value.

Consider the following cases:

- (i) All samples belong to the first class. For this case, assume that we have ideal probabilities that need to be learned by the network. In this case, we also get the ideal entropy.
- (ii) A balanced dataset is given. For this case, assume that we have ideal probabilities that need to be learned by the network. What is the requirement for m so as to have a balanced dataset in each batch?
- (iii) A balanced dataset is given. For this case, assume two classes. Initially, prior to learning, the probabilities are at their worst. You need to give the worst possible probabilities for this case and the corresponding cross-entropy value.

Solution

- (i) Since all samples belong to the first class, the ground-truth vector for every sample is

$$y^{(i)} = [1, 0, \dots, 0]$$

In the ideal case, the network predicts

$$\hat{p}^{(i)} = [1, 0, \dots, 0]$$

Since the correct class probability is 1,

$$-\log(1) = 0$$

Therefore,

$$J(\Theta) = 0$$

- (ii) For a balanced dataset with K classes, the batch size m must satisfy

$$m \bmod K = 0$$

so each class appears equally often. The ground-truth vectors are one-hot encoded, and in the ideal case the predicted probabilities match them exactly:

$$y^{(i)} = [1, 0, 0] \quad \hat{p}^{(i)} = [1, 0, 0]$$

Therefore,

$$J(\Theta) = 0$$

- (iii) For a balanced dataset with two classes, the worst prediction occurs when the network assigns probability 0 to the correct class and 1 to the incorrect class:

$$y^{(i)} = [1, 0] \quad \hat{p}^{(i)} = [0, 1]$$

$$y^{(i)} = [0, 1] \quad \hat{p}^{(i)} = [1, 0]$$

In this case,

$$-\log(0) \rightarrow \infty$$

Therefore, the cross-entropy loss approaches infinity.

Problem #2. Binary Loss functions (15 points).

2(a) Recall the logistic regression loss function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(p^{(i)}) + (1 - y^{(i)}) \log(1 - p^{(i)})]$$

Here, $p^{(i)}$ is the predicted probability and $y^{(i)}$ is the ground-truth label for the i -th sample.

Assume that an object is classified as positive when the positive probability is > 0.5 . For each case below, provide values for all symbols in the loss function.

- (i) A true positive with a positive classification probability of 0.9.
- (ii) A false positive with a negative classification probability of 0.3.
- (iii) A true negative with a positive classification probability of 0.2.
- (iv) A false negative with a positive classification probability of 0.45.

Solution

- (i) True positive:

$$y^{(i)} = 1 \quad p^{(i)} = 0.9$$

$$J(\theta) = -\log(0.9)$$

- (ii) False positive:

$$y^{(i)} = 0 \quad p^{(i)} = 0.7$$

since the negative probability is 0.3.

$$J(\theta) = -\log(0.3)$$

- (iii) True negative:

$$y^{(i)} = 0 \quad p^{(i)} = 0.2$$

$$J(\theta) = -\log(0.8)$$

- (iv) False negative:

$$y^{(i)} = 1 \quad p^{(i)} = 0.45$$

$$J(\theta) = -\log(0.45)$$

2(b) We often get an imbalanced dataset, and we are asked to use weighting factors to rebalance the error. For example, for two classes, if we have 10 times as many samples in class 0, we would multiply the error for class 1 by 10. Provide the modified loss function for this case. This problem refers to the cross-entropy function (see Problem 1).

Solution

For a weighted binary cross-entropy loss, a weighting factor is added to increase the penalty for the minority class. Since class 1 has fewer samples, its error term is multiplied by 10:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [10 y^{(i)} \log(p^{(i)}) + (1 - y^{(i)}) \log(1 - p^{(i)})]$$

This increases the contribution of class 1 errors during training and helps rebalance the learning process.

2(c) Generalize your answer in 2(b) for K classes. This problem refers to the cross-entropy function (see Problem 1).

Solution

For K classes, a separate weighting factor w_k is assigned to each class to account for class imbalance. The weighted cross-entropy loss becomes

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K w_k y_k^{(i)} \log \left(\hat{p}_k^{(i)} \right)$$

where w_k is the weight associated with class k . Larger weights are assigned to minority classes so that their errors contribute more strongly during training.

Problem #3. Basics of Stochastic Gradient Descent (10 points).

3(a) Consider the following relates model that relates the inputs to the outputs:

$$\hat{y}^{(i)} = a \cdot x_{(i)}^2 + b \cdot x_{(i)} + c$$

where i refers to the i -th sample, $\hat{y}^{(i)}$ refers to the estimated ground truth for the i -th sample, $x_{(i)}$ refers to the input value, and a, b refer to the model parameters.

To implement stochastic gradient descent (SGD), we identify that we need to estimate the parameters

$$\theta = (a, b, c).$$

In order to estimate θ , we consider the mean-squared error as a loss function given by:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{i=N} (y^{(i)} - \hat{y}^{(i)})^2.$$

In SGD, instead of evaluating the whole sum, we simply evaluate the error at a single sample:

$$\text{MSE}_1 = (y^{(i)} - \hat{y}^{(i)})^2.$$

Compute the gradient of the single-sample error by substituting the expression for $\hat{y}^{(i)}$ and taking the partial derivatives:

$$\frac{\partial}{\partial a} \text{MSE}_1, \quad \frac{\partial}{\partial b} \text{MSE}_1, \quad \text{and} \quad \frac{\partial}{\partial c} \text{MSE}_1.$$

Then provide the final expression for the gradient using:

$$\nabla_{\theta} \text{MSE}_1 = \left(\frac{\partial}{\partial a} \text{MSE}_1, \frac{\partial}{\partial b} \text{MSE}_1, \frac{\partial}{\partial c} \text{MSE}_1 \right).$$

Hint: While the notation may look intimidating, this is a very simple derivative problem. The following part is also simple because it asks you to evaluate the derivative at a point.

Solution

First, substitute the expression for $\hat{y}^{(i)}$ into the single-sample error:

$$\text{MSE}_1 = (y^{(i)} - (a x_{(i)}^2 + b x_{(i)} + c))^2$$

Using the chain rule, the partial derivatives are:

$$\frac{\partial}{\partial a} \text{MSE}_1 = -2x_{(i)}^2 (y^{(i)} - \hat{y}^{(i)})$$

$$\frac{\partial}{\partial b} \text{MSE}_1 = -2x_{(i)} (y^{(i)} - \hat{y}^{(i)})$$

$$\frac{\partial}{\partial c} \text{MSE}_1 = -2 (y^{(i)} - \hat{y}^{(i)})$$

Therefore, the gradient vector is

$$\nabla_{\theta} \text{MSE}_1 = (-2x_{(i)}^2 (y^{(i)} - \hat{y}^{(i)}), -2x_{(i)} (y^{(i)} - \hat{y}^{(i)}), -2(y^{(i)} - \hat{y}^{(i)}))$$

3(b) We present the modified stochastic gradient descent (SGD) algorithm below. It uses MSE_1 to search for a solution. In what follows, we will investigate a few steps of the algorithm.

Initialize $\theta = (a, b, c)$ using random sampling.

for epoch= 1 to NumOfEpochs

for sample= 1 to NumOfTrainSamples

 Select i using random sampling

Compute weights gradient using (x_i, y_i) :

$$g = \nabla_{\theta} \text{MSE}_1$$

Compute learning rate:

$$\eta = f(\text{epoch}, \text{sample})$$

Update weights using gradient:

$$\theta = \theta - \eta \cdot g$$

In this part, we will trace the application of the algorithm. After random sampling, we set an initial value for

$$\theta = (a, b, c) = (1, 2, 5).$$

For the first epoch, suppose that we randomly sample $i = 3$ and we pick the pair (x_3, y_3) . For this case, let y_3 denote the ideal value for $x_3 = 3$. Assume that

$$\eta = 1e^{-6},$$

and provide an estimate for θ by applying

$$\theta = \theta - \eta \cdot g.$$

Solution

For the sampled point, we have

$$x_3 = 3$$

and the current parameter vector is

$$\theta = (a, b, c) = (1, 2, 5).$$

Using the model,

$$\hat{y}^{(3)} = a x_3^2 + b x_3 + c$$

we obtain

$$\hat{y}^{(3)} = 1(3^2) + 2(3) + 5 = 9 + 6 + 5 = 20.$$

Since y_3 is the ideal value for $x_3 = 3$, the error term is

$$y_3 - \hat{y}^{(3)} = 3 - 20 = -17.$$

Using the gradient expressions from Part 3(a):

$$\frac{\partial}{\partial a} \text{MSE}_1 = -2(3^2)(-17) = 306$$

$$\frac{\partial}{\partial b} \text{MSE}_1 = -2(3)(-17) = 102$$

$$\frac{\partial}{\partial c} \text{MSE}_1 = -2(-17) = 34$$

Therefore,

$$g = (306, 102, 34).$$

Using

$$\eta = 10^{-6},$$

the parameter update becomes

$$\theta = \theta - \eta g$$

$$\theta = (1, 2, 5) - 10^{-6}(306, 102, 34)$$

$$\theta = (0.999694, 1.999898, 4.999966).$$

Problem #4. SoftMax and dot products (20 points).

For the SoftMax layer, we learn parameters $\theta^{(k)}$. Then we compute dot products against the input vectors given by

$$s_k(\mathbf{x}) = (\theta^{(k)})^T \mathbf{x}.$$

The probabilities for each class are given by the softmax function:

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}.$$

We classify the input based on the class that maximizes $\hat{p}_k$ as given by:

$$\hat{y} = \arg \max_k \sigma(\mathbf{s}(\mathbf{x}))_k = \arg \max_k s_k(\mathbf{x}) = \arg \max_k ((\theta^{(k)})^T \mathbf{x})$$

In what follows, we will consider the case of two classes.

4(a) For each of the following cases, sketch how the 2-D input vectors $\mathbf{x}$ will get classified into class 1 or class 2:

(i)

$$\theta^{(1)} = [1, 2]^T, \quad \theta^{(2)} = [-1, 2]^T$$

(ii)

$$\theta^{(1)} = [1, 5]^T, \quad \theta^{(2)} = [5, -1]^T$$

(iii)

$$\theta^{(1)} = [1, 2]^T, \quad \theta^{(2)} = [2, 4]^T$$

For full credit, you need to show:

- (a) all decision vectors plotted.
- (b) shaded region on the 2D plane for each class.
- (c) boundary (if any) between the two classes.

Solution

A point $\mathbf{x} = [x_1, x_2]^T$ is classified by comparing

$$(\theta^{(1)})^T \mathbf{x} \quad \text{and} \quad (\theta^{(2)})^T \mathbf{x}.$$

The decision boundary occurs when

$$(\theta^{(1)} - \theta^{(2)})^T \mathbf{x} = 0.$$

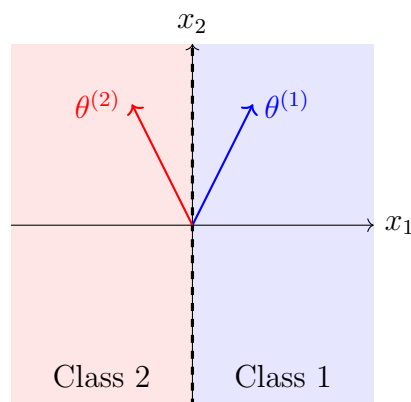
(i)

$$\theta^{(1)} - \theta^{(2)} = [1, 2]^T - [-1, 2]^T = [2, 0]^T$$

so the boundary is

$$2x_1 = 0 \Rightarrow x_1 = 0.$$

Class 1 occurs when $x_1 > 0$, and class 2 occurs when $x_1 < 0$.



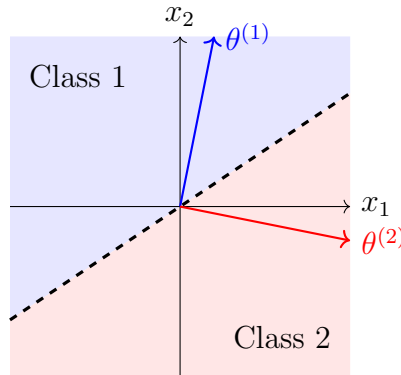
(ii)

$$\theta^{(1)} - \theta^{(2)} = [1, 5]^T - [5, -1]^T = [-4, 6]^T$$

so the boundary is

$$-4x_1 + 6x_2 = 0 \Rightarrow x_2 = \frac{2}{3}x_1.$$

Class 1 occurs when $-4x_1 + 6x_2 > 0$, and class 2 occurs when $-4x_1 + 6x_2 < 0$.



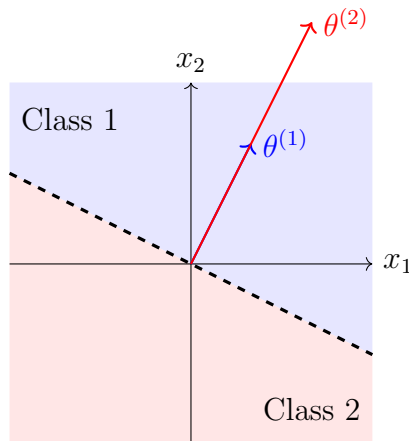
(iii)

$$\theta^{(1)} - \theta^{(2)} = [1, 2]^T - [2, 4]^T = [-1, -2]^T$$

so the boundary is

$$-x_1 - 2x_2 = 0 \Rightarrow x_2 = -\frac{1}{2}x_1.$$

Class 1 occurs when $-x_1 - 2x_2 > 0$, and class 2 occurs when $-x_1 - 2x_2 < 0$.



4(b) For each case given in 4(a), plug-in the ideal input $\mathbf{x} = \theta^{(1)}$ and compute the following:

- (i) Calculate the two probabilities for this case.
- (ii) Apply the probability rule to see what happens. Do you get the correct result?

Solution

For each case, use

$$s_k(\mathbf{x}) = (\theta^{(k)})^T \mathbf{x}, \quad \hat{p}_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2}}.$$

- (i) For $\mathbf{x} = \theta^{(1)} = [1, 2]^T$:

$$\begin{aligned} s_1 &= 5, & s_2 &= 3 \\ \hat{p}_1 &= \frac{e^5}{e^5 + e^3} \approx 0.881, & \hat{p}_2 &= \frac{e^3}{e^5 + e^3} \approx 0.119 \end{aligned}$$

Since $\hat{p}_1 > \hat{p}_2$, the sample is classified as class 1, which is correct.

- (ii) For $\mathbf{x} = \theta^{(1)} = [1, 5]^T$:

$$\begin{aligned} s_1 &= 26, & s_2 &= 0 \\ \hat{p}_1 &= \frac{e^{26}}{e^{26} + e^0} \approx 1, & \hat{p}_2 &= \frac{e^0}{e^{26} + e^0} \approx 0 \end{aligned}$$

Since $\hat{p}_1 > \hat{p}_2$, the sample is classified as class 1, which is correct.

- (iii) For $\mathbf{x} = \theta^{(1)} = [1, 2]^T$:

$$\begin{aligned} s_1 &= 5, & s_2 &= 10 \\ \hat{p}_1 &= \frac{e^5}{e^5 + e^{10}} \approx 0.0067, & \hat{p}_2 &= \frac{e^{10}}{e^5 + e^{10}} \approx 0.9933 \end{aligned}$$

Since $\hat{p}_2 > \hat{p}_1$, the sample is classified as class 2, so the result is incorrect.

Problem #5. Attention equations (10 points).

5(a) For the attention module, we first compute:

$$q_i = x_i W^Q, \quad k_j = x_j W^K, \quad v_j = x_j W^V.$$

Then, the score function is just a dot-product:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}.$$

The key concept here is that we relate queries to keys via a simple dot product between vectors. The standard equations produce row vectors of size $1 \times n$ where n is the number of elements in the flattened x_i, x_j vectors.

Now, for this problem, we want to use Linear Algebra, where the dot product between two column vectors are given by:

$$q_i \cdot k_j = q_i k_j^T.$$

For transposition, recall the basic rules:

$$(A_1 A_2 \cdots A_m)^T = A_m^T A_{m-1}^T \cdots A_2^T A_1^T \quad \text{and} \quad (A^T)^T = A.$$

Substitute the equation for k_j^T to show that:

$$\text{score}(x_i, x_j) = \frac{1}{\sqrt{d_k}} x_i (W^Q (W^K)^T) x_j^T.$$

Solution

Starting with

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}, \quad q_i \cdot k_j = q_i k_j^T$$

and substituting

$$q_i = x_i W^Q, \quad k_j = x_j W^K,$$

we obtain

$$\text{score}(x_i, x_j) = \frac{1}{\sqrt{d_k}} (x_i W^Q) k_j^T.$$

Using the transpose rule,

$$k_j^T = (x_j W^K)^T = (W^K)^T x_j^T.$$

Therefore,

$$q_i k_j^T = (x_i W^Q) ((W^K)^T x_j^T) = x_i (W^Q (W^K)^T) x_j^T.$$

Thus,

$$\text{score}(x_i, x_j) = \frac{1}{\sqrt{d_k}} x_i (W^Q (W^K)^T) x_j^T.$$

5(b) Unlike our SoftMax example, where the inputs are multiplied against fixed vectors, both vectors in the product can change during inference. However, during inference, the weight matrices W^Q and W^K remain fixed to the values that were learned during training. Instead of using the scores to visualize varying relationships based on specific inputs, we can use the product matrix

$$W = W^Q(W^K)^T$$

to visualize permanent relationships based on the locations of the input tokens (e.g., the indices i, j). For the following examples, compute simplified expressions for $\text{score}(x_i, x_j)$:

(i) First row of W is zero.

(ii) First column of W is zero.

Solution

Using the result from 5(a),

$$\text{score}(x_i, x_j) = \frac{1}{\sqrt{d_k}} x_i W x_j^T.$$

Since $x_i W x_j^T$ selects the (i, j) -th entry of W ,

$$\text{score}(x_i, x_j) = \frac{1}{\sqrt{d_k}} W_{ij}.$$

(i) If the first row of W is zero, then

$$W_{1j} = 0 \quad \text{for all } j.$$

Therefore,

$$\text{score}(x_1, x_j) = \frac{1}{\sqrt{d_k}} W_{1j} = 0 \quad \text{for all } j.$$

(ii) If the first column of W is zero, then

$$W_{i1} = 0 \quad \text{for all } i.$$

Therefore,

$$\text{score}(x_i, x_1) = \frac{1}{\sqrt{d_k}} W_{i1} = 0 \quad \text{for all } i.$$

Problem 6. Transformer architecture problem (35 points).

For this problem, you need to provide a system diagram that illustrates the connections between the components. Here are the rules:

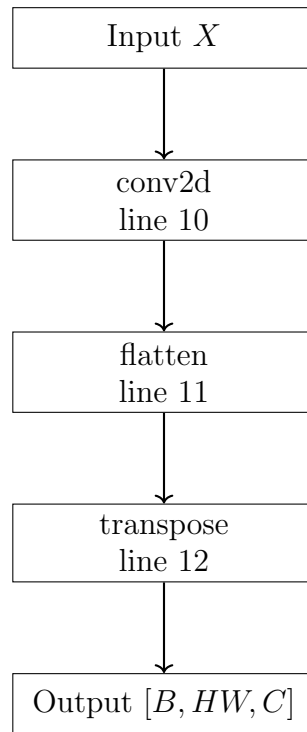
- (i) Use directional arrows to show connections. The beginning of the arrow denotes the input. The end arrow shows where the input is connected.
- (ii) Clearly identify the inputs to the entire system. They should be associated with the start of an arrow.
- (iii) Clearly identify the outputs of the entire system. They should be associated with an arrow ending.
- (iv) Clearly identify the connections of internal variables that appear in the code. This will help me assess that you understand the code.
- (v) Clearly identify skip connections with directional arrows.
- (vi) Label each component using the name of the function and the line number at which it appears in the code. This will help me clearly identify any mistakes that you may be making in the diagram.
- (vii) Treat external function calls as components. In other words, there is no need to go inside and describe the internal connections of external functions. Here, we say that a function is external if it belongs to another class. However, you must clearly identify the connections to the input variables and outputs that appear in the definitions of the functions.
- (viii) You need to describe connections of any internal functions that appear in the class that you are given.
- (ix) Use a simple rectangle to represent each component. Note that dropout needs to appear inside the component where it is applied. It is not a separate component.

Here is the list of classes for which you need to draw a diagram:

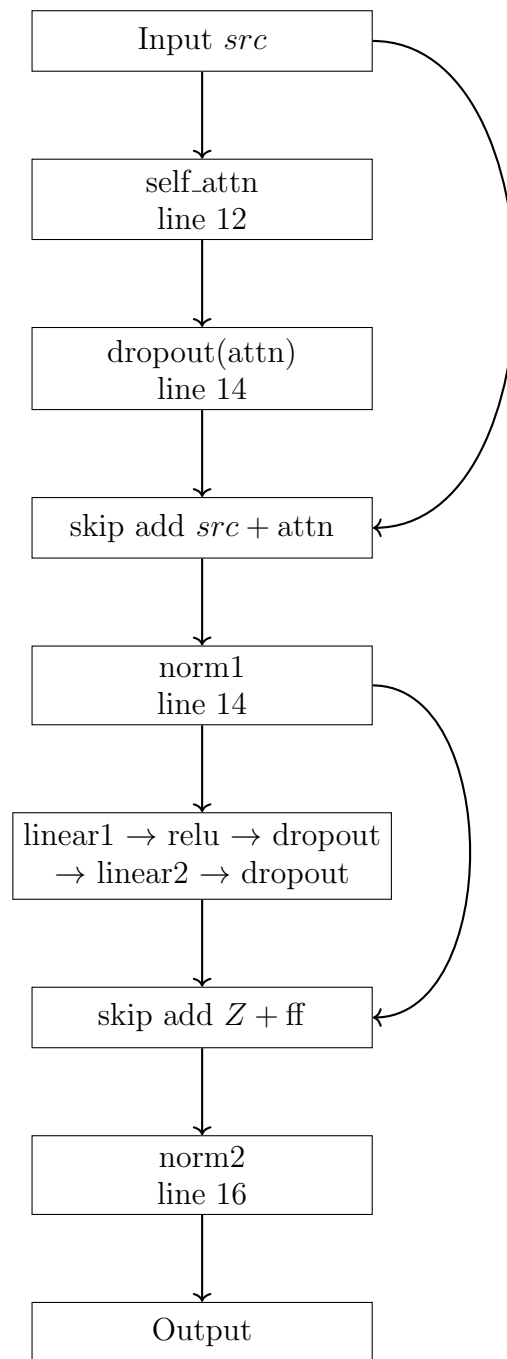
- (a) PatchEmbedding (Fixed_16_vision_and_multimodal_transformers.ipynb)
- (b) TransformerEncoderLayer (Part_15_transformers_for_nlp_and_chatbots.ipynb)
- (c) TransformerEncoder (Part_15_transformers_for_nlp_and_chatbots.ipynb)
- (d) ViT (Fixed_16_vision_and_multimodal_transformers.ipynb)

Solution

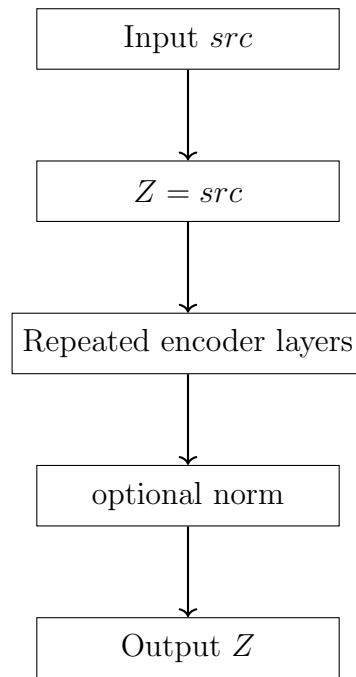
(a) PatchEmbedding



(b) TransformerEncoderLayer



(c) TransformerEncoder



(d) ViT

